

Тема урока: атрибуты объектов и классов

1. ООП в Python
2. Функция `dir()`
3. Атрибуты объектов
4. Атрибуты классов
5. Атрибут `__dict__`
6. Функции `getattr()`, `setattr()`, `hasattr()` и `delattr()`

Аннотация. Урок посвящен атрибутам объектов и классов в Python.

ООП в Python

Язык программирования Python появился в 1991 году. К этому времени ООП уже становилось популярным и появлялись первые объектно-ориентированные языки программирования. Поэтому, ориентируясь на чужие успехи и неудачи, Гвидо ван Россум и его коллеги смогли спроектировать достаточно простую и мощную реализацию ООП в Python. Python поддерживает ООП на сто процентов: все данные в нем являются объектами. Числа, строки, списки, словари, функции, модули и даже сами типы данных — все это объекты. Поэтому их можно присваивать переменным, помещать в списки, хранить в словарях, передавать в функции в качестве аргументов и т. д.



Гвидо ван Россум разработал язык Python по принципу "всё является объектом".

Подробнее об этом можно почитать в его блоге по [ссылке \(http://python-history.blogspot.com/2009/02/first-class-everything.html\)](http://python-history.blogspot.com/2009/02/first-class-everything.html).

Хотя Python — это объектно-ориентированный язык, в предыдущих курсах мы намеренно избегали рассмотрения ООП. Мы убеждены, что проще и интереснее начать изучение Python, не зная обо всех тонкостях объектно-ориентированного программирования, несмотря на то, что оно всегда было рядом с нами.

Приведенный ниже код:

```
num = 10
flag = True
text = '00P'

print(type(num))
print(type(flag))
print(type(text))
```

ВЫВОДИТ:

```
<class 'int'>
<class 'bool'>
<class 'str'>
```

Приведенный ниже код:

```
import math

def cube(num):
    return num ** 3

print(type(math))
print(type(cube))
print(type(print))
print(type(math.factorial))
print(type(str.upper))
print(type(int))
```

ВЫВОДИТ:

```
<class 'module'>
<class 'function'>
<class 'builtin_function_or_method'>
<class 'builtin_function_or_method'>
<class 'method_descriptor'>
<class 'type'>
```

Как мы видим, в Python всё является объектом, принадлежащим соответствующему классу.

Функция dir()

В предыдущих курсах мы нередко сталкивались с типами данных, объекты которых имеют различные атрибуты и методы. Например, объекты типа `date` из модуля `datetime` обладают немалым количеством полезных атрибутов и методов.



Тип данных и класс — это одно и то же.

Приведенный ниже код:

```
from datetime import date

today = date(2022, 9, 25)

print(today.year)           # год
print(today.month)          # месяц
print(today.day)            # день
print(today.isoweekday())    # порядковый номер дня недели
print(today.toordinal())     # номер дня, соответствующий дате 2022-09-25
```

ВЫВОДИТ:

```
2022
9
25
7
738423
```

В примере выше `year`, `month` и `day` являются атрибутами, а `isoweekday()` и `toordinal()` — методами.

Для получения списка всех атрибутов и методов объекта можно воспользоваться встроенной функцией `dir()`.

Приведенный ниже код:

```
from datetime import date

today = date(2022, 9, 25)

print(dir(today))
```

ВЫВОДИТ:

```
['__add__', '__class__', '__delattr__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__getattribute__', '__gt__', '__hash__', '__init__',
 '__init_subclass__', '__le__', '__lt__', '__ne__', '__new__', '__radd__',
 '__reduce__', '__reduce_ex__', '__repr__', '__rsub__', '__setattr__', '__sizeof__',
 '__str__', '__sub__', '__subclasshook__', 'ctime', 'day', 'fromisocalendar',
 'fromisoformat', 'fromordinal', 'fromtimestamp', 'isocalendar', 'isoformat',
 'isoweekday', 'max', 'min', 'month', 'replace', 'resolution', 'strftime',
 'timetuple', 'today', 'toordinal', 'weekday', 'year']
```

Создание пользовательских классов и их объектов

В Python классы создаются с помощью инструкции `class`, за которой следует имя класса, после которого ставится двоеточие. Далее с новой строки и с отступом реализуется тело класса:

```
class <название класса>:
    <тело класса>
```

Пример объявления класса с минимально возможным функционалом:

```
class Cat:  
    pass
```

Здесь для задания класса используется инструкция `class`, далее следует имя класса `Cat` и двоеточие. После идет тело класса, которое в нашем случае представлено оператором `pass`, который сам по себе ничего не делает и является просто заглушкой.

Как правило, имя класса начинается с заглавной буквы и обычно является существительным или словосочетанием. Имена классов соответствуют соглашению Upper Camel Case.



Upper Camel Case (UpperCamelCase) — стиль написания составных слов, при котором несколько слов пишутся слитно без пробелов, при этом каждое слово пишется с заглавной буквы.

Чтобы создать объект класса, нужно воспользоваться следующим синтаксисом:

```
<имя переменной> = <имя класса>()
```

В качестве примера создадим объект класса `Cat` и сохраним его в переменную `cat`:

```
cat = Cat()
```



Объект класса также называют экземпляром класса. Это одно и то же, однако для однозначности обычно используют термин экземпляр класса, так как сам класс тоже является объектом. В рамках курса мы будем использовать оба термина.

Атрибуты объектов

Вернемся к созданному ранее классу `Cat` и создадим два его экземпляра:

```
class Cat:  
    pass  
  
cat1 = Cat()  
cat2 = Cat()
```

Переменные `cat1` и `cat2` содержат ссылки на два разных объекта — экземпляра класса `Cat`, которые можно наделить различными атрибутами:

```
cat1.breed = 'Британский'      # порода кошки
cat1.name = 'Кемаль'          # имя кошки
cat1.age = 1                   # возраст кошки

cat2.name = 'Роджер'
cat2.age = 5
```

Теперь `cat1` — это британский кот с именем Кемаль, которому 1 год, а `cat2` — кот с именем Роджер, которому 5 лет. Важно заметить, что указанные атрибуты принадлежат конкретному экземпляру класса `Cat`. Например, `cat2` не содержит атрибута `breed`, который есть у `cat1`.

К установленным объекту атрибутам можно обращаться, а также изменять их.

Приведенный ниже код:

```
cat = Cat()

cat.breed = 'Британский'
cat.name = 'Кемаль'
cat.age = 1

print(cat.breed, cat.name)      # обращение к атрибутам

cat.age += 2                     # изменение значения атрибута
print(cat.age)
```

ВЫВОДИТ:

```
Британский Кемаль
3
```

При обращении к несуществующему атрибуту будет возбуждено исключение `AttributeError`.

Приведенный ниже код:

```
cat = Cat()

cat.breed = 'Британский'
cat.name = 'Кемаль'
cat.age = 1

print(cat.color)
```

приводит к возбуждению исключения:

```
AttributeError: 'Cat' object has no attribute 'color'
```

Атрибуты класса

Мы также можем устанавливать атрибуты на уровне всего класса, такие атрибуты называются **атрибутами класса**. Их значения одинаковы для всех экземпляров этого класса:

```
class Cat:
    night_vision = True
    paws_count = 4
```

Теперь класс `Cat` имеет два атрибута, доступных по умолчанию всем его экземплярам.

Приведенный ниже код:

```
class Cat:
    night_vision = True
    paws_count = 4

cat1 = Cat()
cat2 = Cat()

print(cat1.night_vision)
print(cat2.paws_count)
```

ВЫВОДИТ:

```
True
4
```

Так как атрибуты `night_vision` и `paws_count` являются атрибутами всего класса, то мы можем получить к ним доступ, используя точечную запись с именем класса.

Приведенный ниже код:

```
class Cat:
    night_vision = True
    paws_count = 4

print(Cat.night_vision)
print(Cat.paws_count)
```

ВЫВОДИТ:

```
True
4
```

Как правило, названия атрибутов объектов и классов пишутся с маленькой буквы и соответствуют соглашению snake case.



Snake Case (snake_case) — стиль написания составных слов, при котором несколько слов разделяются символом нижнего подчеркивания (`_`) и не имеют пробелов в записи, причём каждое слово пишется с маленькой буквы.

Атрибут `__dict__`

Все атрибуты, которыми мы наделяем созданные объекты, хранятся в специальном словаре, который доступен в качестве атрибута `__dict__`.

Приведенный ниже код:

```
class Cat:
    pass

cat = Cat()

cat.breed = 'Британский'
cat.name = 'Кемаль'
cat.age = 1

print(cat.__dict__)
```

ВЫВОДИТ:

```
{'breed': 'Британский', 'name': 'Кемаль', 'age': 1}
```

Аналогичным образом мы можем получить доступ к атрибутам класса.

Приведенный ниже код:

```
class Cat:
    night_vision = True
    paws_count = 4

print(Cat.__dict__)
```

ВЫВОДИТ:

```
{'__module__': '__main__', 'night_vision': True, 'paws_count': 4, '__dict__':
<attribute '__dict__' of 'Cat' objects>, '__weakref__': <attribute '__weakref__' of
'Cat' objects>, '__doc__': None}
```

Обратите внимание, что помимо созданных нами атрибутов класса `night_vision` и `paws_count`, словарь `__dict__` также содержит и другие служебные атрибуты, в том числе описание самого словаря `__dict__`.



Атрибуты объектов и классов, которые определил программист, называются **пользовательскими атрибутами**.

Примечания

Примечание 1. ООП — один из самых мощных инструментов Python, однако нам необязательно его использовать. Мы можем писать мощные и эффективные программы и без него.

Примечание 2. Имена классов по стандарту именования PEP 8 должны соответствовать соглашению UpperCamelCase, однако встроенные классы (`int`, `float`, `str`, `list`, `tuple`, `dict` и др.) этому правилу не следуют.

Примечание 3. Аналогично функциям, классы поддерживают строку документации, которая доступна через атрибут `__doc__`.

Приведенный ниже код:

```
class Cat:
    """Класс, описывающий кошку"""
    night_vision = True
    paws_count = 4

print(Cat.__doc__)
```

ВЫВОДИТ:

```
Класс, описывающий кошку
```

Примечание 4. Прочитать подробнее о разных стилях именования можно по [ссылке](https://khalilstemmler.com/blogs/camel-case-snake-case-pascal-case/) (<https://khalilstemmler.com/blogs/camel-case-snake-case-pascal-case/>).

Примечание 5. Прочитать про соглашения об именах в Python можно на русском (<https://docs-python.ru/tutorial/pep-rukovodstvo-stilju-koda-python/soglashenija-imenah/>) и английском (https://visualgit.readthedocs.io/en/latest/pages/naming_convention.html) языках.

Примечание 6. Тип любого объекта можно получить через атрибут `__class__`.

Приведенный ниже код:


```
class Cat:
    pass

cat = Cat()

print(type(cat))
print(cat.__class__)
print(type(cat) == cat.__class__)
```

ВЫВОДИТ:

```
<class '__main__.Cat'>
<class '__main__.Cat'>
True
```

Примечание 7. Атрибут класса можно изменить только через сам класс.

Приведенный ниже код:

```
class Cat:
    night_vision = True

cat1 = Cat()
cat2 = Cat()

Cat.night_vision = False

print(cat1.night_vision)
print(cat2.night_vision)
```

ВЫВОДИТ:

```
False
False
```

При попытке изменения атрибута класса через его экземпляр мы лишь добавляем этому экземпляру атрибут с аналогичным именем.

Приведенный ниже код:



